

1. El patrón **Adapter** se utiliza principalmente para:
- a) Reducir la cantidad de objetos creados en memoria.
 - b) Convertir la interfaz de una clase existente en otra interfaz que el cliente espera.
 - c) Garantizar que solo exista una instancia de una clase.
 - d) Separar un objeto de su representación textual.
-

2. ¿Cuál de los siguientes patrones es más apropiado para **componer objetos en estructuras jerárquicas** y tratar objetos individuales y compuestos de la misma forma?
- a) **Bridge**
 - b) **Decorator**
 - c) **Composite**
 - d) **Proxy**
-

3. En el patrón **Bridge**, la idea principal es:
- a) Cambiar el comportamiento de un objeto en tiempo de ejecución.
 - b) Aumentar la compatibilidad entre interfaces no relacionadas.
 - c) Separar una abstracción de su implementación para que ambas puedan evolucionar independientemente.
 - d) Permitir que varias clases compartan un estado común.
-

4. Si quieres **añadir responsabilidades adicionales a un objeto en tiempo de ejecución sin modificar su clase original**, el patrón más adecuado es:
- a) **Decorator**
 - b) **Proxy**
 - c) **Adapter**
 - d) **Facade**
-

5. El patrón **Proxy** y el patrón **Decorator** son similares porque ambos:
- a) Requieren múltiples herencias para funcionar.
 - b) Aumentan la compatibilidad entre interfaces distintas.
 - c) Reemplazan herencia con sobrecarga de operadores.
 - d) Se basan en la composición y delegación.